

PSet 2: Pipeline de Datos End-to-End

Fundamentos de Ciencia de Datos

Octubre, 2026

Integrantes: Sebastián Silva, Atik Santellán, Jorge Almeida y Juan José Torres

Entregable	Dirección
Documento Técnico	Este documento.
Arquitectura	Figura 1
Repositorio, Snowflake, Kestra, dbt y Spark	https://github.com/iCebka/elt_soccer_value/tree/main

1: Arquitectura

La arquitectura implementada sigue el flujo de datos desde Fuente a las capas del modelo de medalla, es decir, Bronze, Silver, Gold y se finaliza incluyendo también una OBT. Se emplea Snowflake como almacenamiento central. Kestra orquesta el pipeline completo: descarga los snapshots del dataset, carga los datos originales en la capa Bronze y posteriormente ejecuta las transformaciones de dbt y la construcción de la OBT con Spark. dbt materializa las capas Silver y Gold dentro de Snowflake; Silver concentra la limpieza, tipado y validación de los datos, mientras que Gold organiza la información en dimensiones y tablas de hechos. Finalmente, Spark consume las tablas Gold, unifica los distintos tipos de hechos en *OBT_FOOTBALL_EVENTS* y escribe nuevamente el resultado en Snowflake.

La infraestructura local se define mediante Docker Compose con dos servicios. *kestra-db* ejecuta PostgreSQL y almacena el estado interno, la cola y el repositorio de Kestra. El segundo contenedor ejecuta Kestra y utiliza una imagen personalizada que incorpora dbt Core y PySpark, por lo que ambos se ejecutan como procesos dentro del mismo contenedor. Snowflake permanece externo al entorno Docker y actúa como destino persistente de las capas Bronze, Silver, Gold y de la OBT.

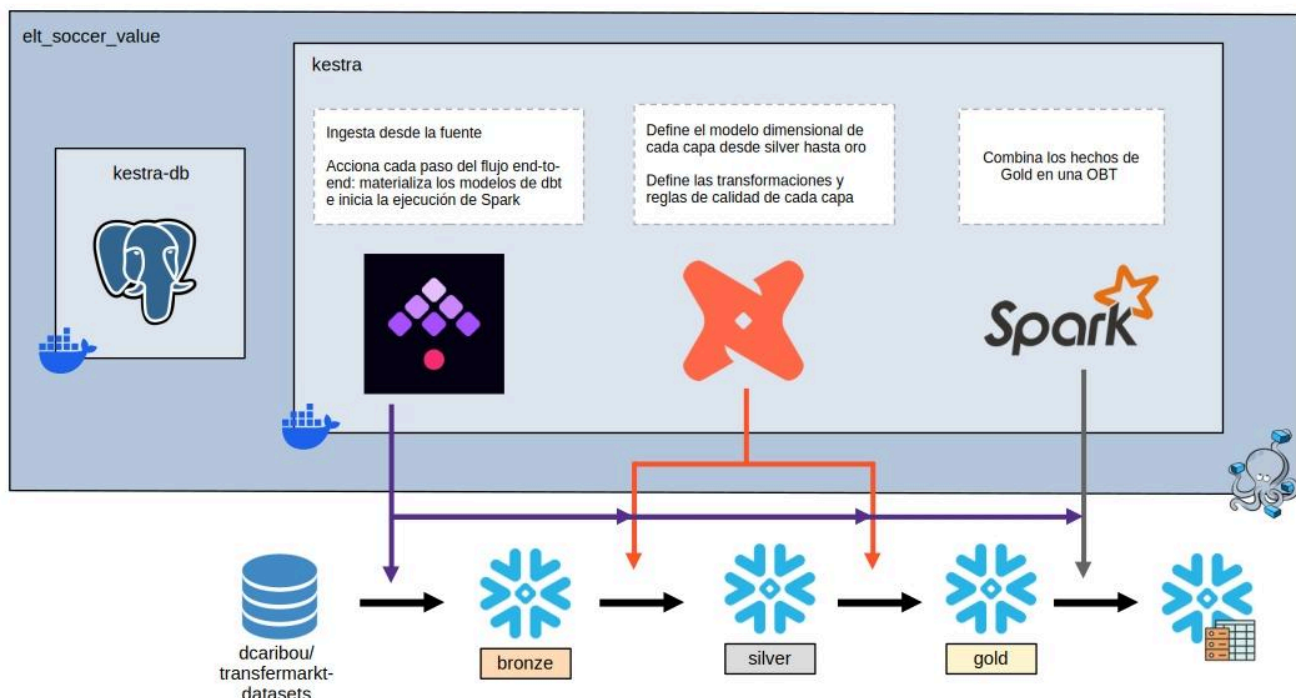


Figura 1. Arquitectura end-to-end del pipeline ELT. Docker Compose contiene los servicios locales de orquestación y ejecución, mientras que Snowflake almacena las distintas capas del modelo de datos.

2: Ingesta con Kestra

La fuente utilizada es el endpoint público provisto por el autor del dataset <https://github.com/dcaribou/transfermarkt-datasets>. Cada archivo del repositorio es un snapshot completo de una tabla, sus eventos históricos conservan su fecha. El trigger utilizado está configurado para ejecutarse la actualización los miércoles y sábados a las 08:00 en America/Guayaquil. Este trigger es independiente de la actualización de la base de datos. El repositorio asociado al dataset ha reportado que desde julio de este año se detiene la actualización recurrente que se realizaba hasta nuevo aviso. Se mantiene este trigger recurrente como ejercicio didáctico.

Archivo, Relación	Granularidad
competitions, clubs, players	Competición, club o jugador
countries, national_teams	País y selección nacional
games	Partido identificado por game_id
appearances	Aparición de un jugador en un partido
club_games	Club en un partido identificado por game_id y club_id
game_events	Acción registrada en un partido con id game_event_id
game_lineups	Registro de alineación para un partido con id game_lineups_id
player_valuations	Valoración de un jugador con fecha registrada
transfers	Movimiento de jugador entre clubes

Cargas y retries: Kestra descarga cada CSV comprimido y lo carga en TRANSFERMARKT_STAGE bajo un prefijo asociado al execution.id. A partir de ese archivo crea una tabla temporal <asset>_LOAD, ejecuta COPY INTO por nombre de columna y añade los metadatos _source_file, _source_row_number y _ingested_at. Si la carga termina correctamente, SWAP publica la tabla candidata como nueva versión de Bronze; después se elimina la versión anterior y se limpian tanto el archivo del stage como la descarga local de Kestra. Este mecanismo se eligió porque la fuente publica snapshots completos de cada tabla, no cambios incrementales. Las tareas de descarga y subida al stage utilizan reintentos exponenciales con un intervalo inicial de 30 segundos, un máximo de 2 minutos entre intentos, 3 intentos como máximo y una duración límite de 5 minutos. Las ejecuciones del flujo se encolan con concurrencia uno y los archivos se procesan secuencialmente. El trigger recupera únicamente la última ejecución programada perdida. Las tareas SQL, dbt y Spark no tienen reintentos automáticos propios, de modo que un error de carga, validación o procesamiento hace fallar la ejecución.

Backfill: La fuente distribuye snapshots completos y varias tablas no tienen una dimensión temporal que permita definir un progreso incremental. Por esta razón, la forma habitual de recuperar o actualizar un asset es volver a ejecutar el flujo principal para esa tabla, reemplazando Bronze con el snapshot actual. Esta operación recupera el estado publicado en la fuente, pero no reconstruye cómo era el dataset en una fecha pasada. Para permitir correcciones por intervalo se implementó además backfill_football_it1, que recibe assets, backfill_start y backfill_end. El flujo vuelve a descargar el snapshot actual y, para games, appearances, player_valuations, game_events y game_lineups, reemplaza las filas cuyo campo date se encuentra dentro del intervalo solicitado; transfers utiliza transfer_date. Las filas fuera del intervalo y aquellas cuya fecha no puede interpretarse se conservan. competitions, clubs, players, club_games, countries y national_teams no poseen una fecha de negocio adecuada para este procedimiento, por lo que, si se seleccionan, se reemplaza su snapshot completo. Después de publicar cualquier cambio, el flujo reconstruye Silver, Gold y la OBT completa para mantener consistencia entre las capas. El backfill por intervalo materializa además una tabla candidata

combinando el estado existente con las filas del nuevo snapshot, por lo que se reserva para correcciones históricas concretas. Para la actualización ordinaria, la recarga completa por asset resulta más directa y corresponde mejor con la forma en que se publica la fuente.

3: Calidad y limpieza de datos

Silver conserva los metadatos de Bronze y aplica etapas de parsing, normalización, validación y deduplicación. Las reglas implementadas deben medirse sobre la ejecución identificada antes de reportarlas como problemas observados. Estas son las reglas y decisiones de calidad más relevantes para preparar la base de datos de cara al futuro modelo:

Tabla y campo	Evidencia del perfilado	Decisión y justificación
appearances: player_current_club_id	2 de 1.894.350 registros con -1 (0,0001 %). Sin valores nulos.	Convertir a INTEGER y conservar -1 como etiqueta de sin club. Permitir NULL. Evita tratar un estado especial como un ID inválido y descartar la aparición.
clubs y national_teams: total_market_value	clubs: 796 de 796 nulos (100 %). national_teams: 4 de 124 nulos (3,23 %).	Convertir a DECIMAL y exigir ≥ 0 cuando exista. Mantener NULL y conservar la fila. Imputar cero atribuiría un valor económico que la fuente no informa.
game_events: minute	17.575 de 1.274.469 registros con -1 (1,38 %). Sin valores nulos; los ejemplos incluyen Shootout.	Convertir a INTEGER, mantener NOT NULL y conservar -1 como etiqueta especial; los demás valores deben ser ≥ 0 . Evita perder eventos o inventar su minuto. Se puede interpretar aunque no exclusivamente como tiempo añadido.
players, transfers y player_valuations: market_value_in_eur	players: 8.621 de 50.149 nulos (17,19 %). transfers: 68.160 de 175.165 nulos (38,91 %). player_valuations: 0 nulos y 0 negativos en 656.301 filas.	Convertir a DECIMAL y exigir ≥ 0 . Es obligatorio en player_valuations porque sustenta el objetivo predictivo; en players y transfers se admite NULL. Evitar imputar cero, que es distinto de una valoración desconocida.
players y player_valuations: current_club_id	-1 en clubs clubs: players: 2.986/50.149 (5,95 %). player_valuations: 92.973/656.301 (14,17 %). Sin nulos en ambos campos.	Convertir a INTEGER, permitir NULL y no exigir FK estricta a clubs. Un jugador puede carecer de club y el catálogo cubre solo parte del histórico; estos casos no invalidan el registro. En players, conservar -1 como etiqueta de sin club si aparece.

Para estandarizar los datos se definieron conversiones explícitas a INTEGER, DATE, DECIMAL y BOOLEAN según el tipo de cada campo. Los textos se normalizan con TRIM; los códigos de competición y país se almacenan en mayúsculas y las categorías se normalizan en minúsculas cuando corresponde. Los indicadores 0/1 se convierten a booleanos y net_transfer_record se transforma a euros respetando tanto el signo como los sufijos monetarios de la fuente. Los valores -1 se conservan cuando representan una categoría especial; en current_national_team_id, en cambio, se convierten a NULL.

El perfilado no encontró filas completamente duplicadas. Tampoco se observaron nulos ni repeticiones en las claves seleccionadas: los identificadores principales de las tablas y las claves compuestas (game_id, club_id) en club_games, (player_id, date) en player_valuations y (player_id, transfer_date, from_club_id, to_club_id) en transfers.

En la revisión de integridad referencial se encontraron cero registros huérfanos en player_valuations.player_id y transfers.player_id respecto de players, y en national_teams.country_id respecto de countries; por ello, estas tres relaciones se declararon obligatorias. También se encontraron cero game_id huérfanos en appearances, game_events, game_lineups y club_games respecto de games. La comparación de las fechas de appearances, game_events y game_lineups con la fecha del partido correspondiente cubrió 6.347.835 registros y no encontró discrepancias. El repositorio incluye además un notebook con el perfilado detallado de cada tabla del dataset.

4: Modelado dimensional con dbt

La capa Gold reorganiza los modelos Silver en una constelación de cuatro esquemas estrella con dimensiones compartidas. Se mantuvieron separados los procesos de participación en partidos,

acciones de partido, transferencias y valoraciones porque cada uno representa una observación de negocio diferente. Las dimensiones conformadas dim_player, dim_club, dim_competition y dim_date proporcionan contexto común sin modificar el grain de cada hecho. Gold no repite las reglas de limpieza de Silver; selecciona las observaciones analíticamente válidas y establece explícitamente sus relaciones y granularidades.

Tabla de hechos	Grano	Contenido principal
fact_player_match	(game_id, player_id)	Participación de un jugador en un partido
fact_match_event	game_event_id	Acción registrada dentro de un partido
fact_transfer	(player_id, transfer_date, from_club_id, to_club_id)	Movimiento de un jugador entre clubes
fact_player_valuation	(player_id, valuation_date)	Valor de mercado observado en una fecha

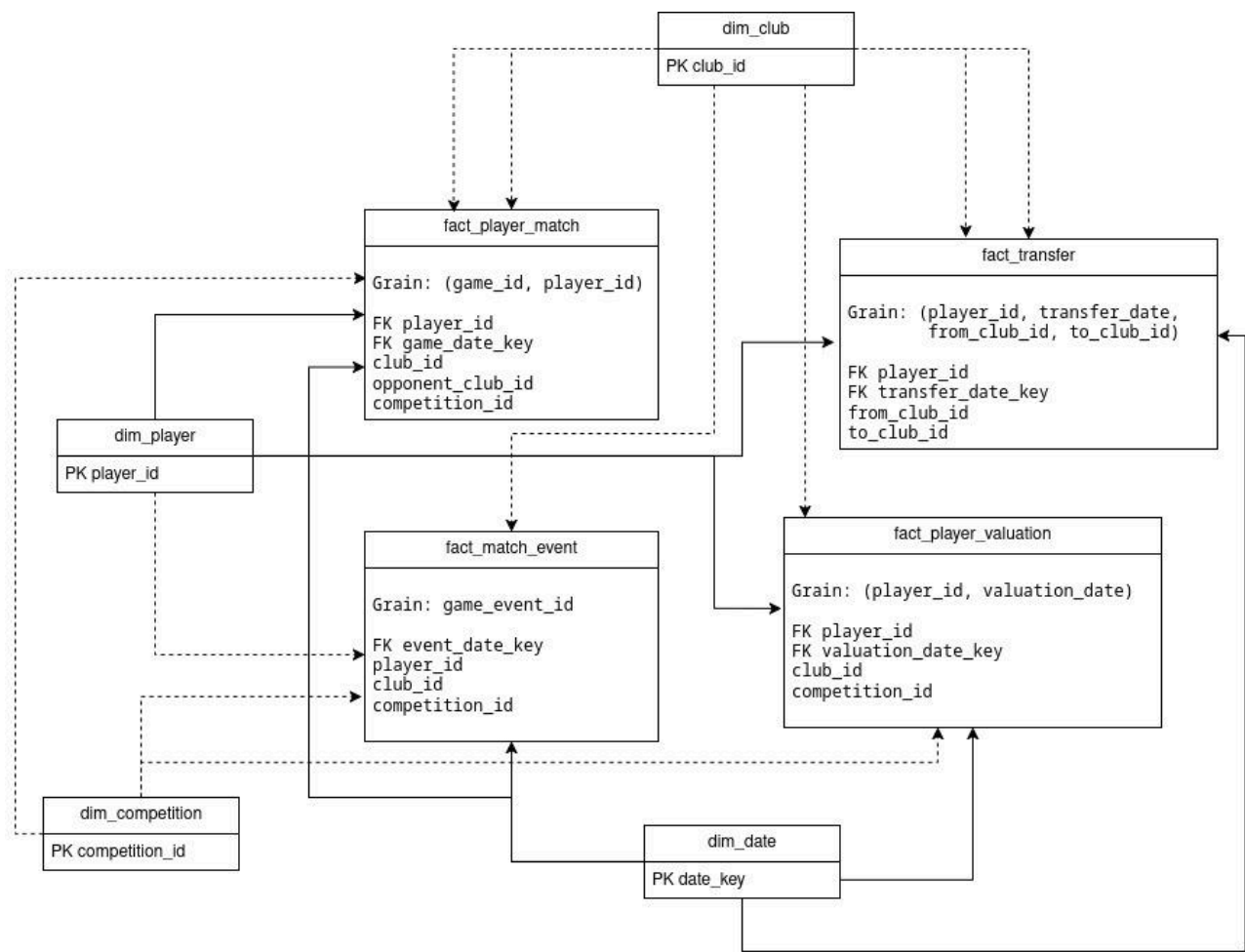


Figura 2. Modelo dimensional Gold. Constelación de cuatro tablas de hechos con dimensiones compartidas de jugador, fecha, club y competición. Las líneas continuas indican relaciones obligatorias y las discontinuas relaciones opcionales, permitiendo conservar hechos históricos aun cuando no exista cobertura completa en las dimensiones descriptivas.

Las dependencias del proyecto se gestionan con source() para referenciar las tablas Bronze y con ref() para enlazar los modelos Silver, dimensiones y hechos de Gold. Los contratos de dbt incluyen pruebas not_null, unique y relationships sobre claves y relaciones obligatorias; además se validan dominios concretos, como los valores de calendario y home_or_away para clubes en partidos.

5: OBT con Spark

La OBT GOLD.OBT_FOOTBALL_EVENTS se construye con Spark a partir de las cuatro tablas de hechos de Gold. Cada hecho se proyecta primero sobre un esquema común y se identifica mediante event_type: PLAYER_MATCH, MATCH_ACTION, TRANSFER o VALUATION. Las cuatro proyecciones se combinan verticalmente mediante unionByName; no se realizan joins entre tablas de hechos, ya que cada una conserva un grano distinto. Los campos que no aplican a un determinado tipo de evento permanecen como NULL.

El grano general de la OBT es una observación temporal correspondiente a uno de esos cuatro procesos. event_type conserva la semántica del grano original y cada fila recibe un event_id determinista generado mediante SHA-256 a partir del tipo de evento y de su clave natural. De esta forma, una fila Gold produce exactamente una fila OBT y puede rastrearse hasta su hecho de origen mediante source_fact y source_key. obt_decisions obt_decisions

Después de la unión, Spark añade el contexto de dim_date, dim_player, dim_club y dim_competition. La cobertura de dim_date es obligatoria; los enriquecimientos descriptivos opcionales se realizan con LEFT JOIN para no eliminar observaciones. El proceso comprueba el número de filas después de cada join y falla si este cambia, evitando que una relación many-to-many introduzca duplicados. obt_football_events

Antes de escribir el resultado se validan los campos comunes obligatorios, los contratos de cada event_type, la unicidad de event_id y los conteos por tipo. Para cada tipo debe cumplirse que el número de filas en la OBT sea igual al número de filas de su tabla de hechos Gold, por lo que el total final debe ser la suma de las cuatro facts. La tabla solo se escribe después de superar estas comprobaciones y se reemplaza completamente mediante overwrite en Snowflake.

event_type	Hecho en Gold	Grano original
PLAYER_MATCH	fact_player_match	(game_id, player_id)
MATCH_ACTION	fact_match_event	game_event_id
TRANSFER	fact_transfer	(player_id, transfer_date, from_club_id, to_club_id)
VALUATION	fact_player_valuation	(player_id, valuation_date)

Conviene señalar que esta OBT no anula el uso del constellation schema definido en Gold. Este se utiliza cuando se necesitan análisis estructurados y flexibles sobre entidades específicas. Esto se puede dar cuando se estudian jugadores, clubes, partidos, transferencias o valoraciones por separado. Al mantener hechos y dimensiones independientes, se puede hacer consultas analíticas más claras y evitar repetir atributos innecesariamente.

La OBT se utiliza cuando se necesita una vista única y desnormalizada para análisis exploratorio, consumo por modelos de machine learning o consultas que requieren combinar repetidamente información de varias tablas. Esto reduce la necesidad de realizar múltiples joins durante el consumo. Por lo tanto, Gold funciona como la capa analítica normalizada y reutilizable, mientras que la OBT funciona como una capa de consumo simplificado para casos en los que sea beneficioso tener toda la información relevante en una sola tabla.

6: Batch y Streaming

El pipeline se diseñó como un sistema de procesamiento batch porque la fuente publica archivos comprimidos que representan snapshots completos de los datasets de *Transfermarkt*. El flujo principal

de Kestra está programado para ejecutarse los miércoles y sábados a las 08:00 (America/Guayaquil); cuando se ejecuta, actualiza los assets seleccionados y reconstruye las capas derivadas Silver, Gold y OBT

Los datos utilizados (partidos, participaciones, transferencias y valoraciones de mercado) tienen un uso principalmente histórico y analítico. Su utilidad para el proyecto no depende de procesarlos en milisegundos, por lo que la latencia entre dos ejecuciones programadas no afecta de forma relevante al objetivo del futuro modelo

Una arquitectura de streaming tendría sentido si los datos llegarán continuamente y existiera una necesidad de reaccionar con baja latencia. En este caso sería necesario introducir mecanismos adicionales para gestionar particiones, offsets, reintentos, duplicados y procesamiento de eventos tardíos, aumentando la complejidad operativa sin resolver una necesidad presente

Esta decisión podría revisarse si en el futuro se incorporarán fuentes como eventos en vivo de partidos, cambios de mercado en tiempo real u otras señales cuya utilidad dependa del momento de recepción. En ese escenario tendría sentido considerar una arquitectura híbrida o de streaming. Incluso entonces, la necesidad dependería del uso final: decisiones como realizar una oferta o ceder un jugador no requieren necesariamente una respuesta en tiempo real. El procesamiento batch se ajusta mejor tanto a la forma en que se publica la fuente como al tipo de análisis histórico planteado en el proyecto

7: Limitaciones

El pipeline cubre el proceso completo desde la ingesta hasta la construcción de la OBT, pero mantiene varias limitaciones. La primera es la frescura de los datos: el sistema no opera en tiempo real y depende tanto de la programación de Kestra como de la frecuencia con la que el proveedor publique nuevos snapshots.

El backfill tampoco permite reconstruir cualquier estado histórico de la fuente. El flujo vuelve a descargar el snapshot disponible en ese momento y utiliza sus filas para reemplazar el intervalo solicitado. Esto permite corregir o volver a cargar periodos que todavía estén representados en la fuente actual, pero no recuperar una versión histórica que el proveedor ya haya sustituido o dejado de publicar.

El alcance del repositorio termina en la preparación de los datos. Todavía no se implementan el entrenamiento, despliegue ni monitoreo del futuro modelo de ciencia de datos. La OBT se diseñó como entrada para esa etapa, pero su esquema podría modificarse cuando las primeras iteraciones del modelo permitan identificar nuevas variables, ventanas temporales o requisitos de información

Existe además una dependencia directa del proyecto transfermarkt-datasets. Actualmente sus actualizaciones automáticas están pausadas y el dataset se declara vigente hasta el 6 de julio de 2026; algunas tablas tienen fechas máximas anteriores, como appearances y player_valuations.

8: Reproducibilidad

La infraestructura local se encapsula mediante Docker. El `Dockerfile` fija las principales dependencias del entorno, incluyendo Kestra, dbt, PySpark y Java, mientras que Docker Compose declara los servicios, volúmenes y configuración necesarios para ejecutar el pipeline. Esto reduce la dependencia de instalaciones manuales en la máquina del usuario y permite reconstruir el entorno a partir de archivos versionados. La reproducción completa sigue dependiendo de servicios externos: se requieren credenciales válidas de Snowflake, acceso a la fuente y disponibilidad de las imágenes y paquetes utilizados durante la construcción. El repositorio incluye la configuración del pipeline y una guía de despliegue con los pasos necesarios para levantar la infraestructura y ejecutar los flujos:

https://github.com/iCebka/elt_soccer_value/tree/main.